

Day 22: Error Handling in Embedded

Platform: nRF52840 | RTOS: Zephyr RTOS

Learning Objectives

Error Codes vs Exceptions

C++ exceptions (`throw / catch`) are **disabled** in embedded (`-fno-exceptions` in Zephyr). The alternatives:

- **Return codes:** return `int` (0 = success, negative = error code). This is the Zephyr convention.
- **Result type:** a struct/template holding value + error code
- **Error callbacks:** call a registered error handler function
- **Panic/assert:** for unrecoverable errors (`k_panic()` , `__ASSERT()`)

static_assert for Compile-Time Checks

`static_assert` fires at **compile time** with a clear message — zero runtime cost:

```
static_assert(sizeof(uint32_t) == 4, "uint32_t must be 4 bytes on this platform");
static_assert(CONFIG_SYS_CLOCK_TICKS_PER_SEC >= 100, "Need at least 100Hz tick");
static_assert((BUFFER_SIZE & (BUFFER_SIZE-1)) == 0, "Buffer size must be power of 2");
```

Use `static_assert` to document and enforce invariants that must be true for the code to work correctly.

__ASSERT() for Debug-Time Checks

`__ASSERT(condition, format, ...)` checks the condition **only in debug builds** (when `CONFIG_ASSERT=y`). In production builds, the entire check is compiled out:

```
__ASSERT(ptr != nullptr, "Null pointer passed to read_sensor()");
__ASSERT(len <= MAX_BUF_SIZE, "Length %zu exceeds buffer %zu", len, MAX_BUF_SIZE);
```

Return Value Patterns

Zephyr convention: return `int` where 0 = success, negative errno = error:

```
int configure_sensor(uint8_t id, uint32_t rate_hz) {
    if (id >= MAX_SENSORS) return -EINVAL; // Invalid argument
    if (rate_hz == 0)      return -EINVAL;
    if (rate_hz > 1000)    return -ENOTSUP; // Not supported
    return 0; // Success
}
// Caller:
int ret = configure_sensor(2, 100);
if (ret < 0) {
    LOG_ERR("Config failed: %d (%s)", ret, strerror(-ret));
}
```

Result Pattern

A `Result` type carries either a success value or an error — like Rust's `Result` :

```
template <typename T>
struct Result {
    T value;
    int err; // 0 = OK, negative = error
    static Result ok(T v) { return {v, 0}; }
    static Result error(int e) { return {{}, e}; }
    bool is_ok() const { return err == 0; }
};
auto result = read_adc(channel);
if (result.is_ok()) { process(result.value); }
```

Zephyr Error Codes (-EINVAL etc.)

Zephyr uses POSIX errno values as negative numbers:

```
-EINVAL    = -22  : Invalid argument
-ENOTSUP   = -134 : Operation not supported
-EBUSY     = -16  : Device/resource busy
```

```
-ETIMEDOUT = -110 : Timed out
-EIO       = -5   : I/O error
-ENOMEM    = -12  : Out of memory
```

Use `strerror(-ret)` to convert to a human-readable string.

Practice Tasks

1. Wrap Zephyr's `gpio_pin_configure_dt()` in a function that returns your `Result` type
 2. Build a chain of function calls where each checks the previous return code and propagates errors up
 3. Create compile-time checks for all your buffer sizes and tick rates using `static_assert`
 4. Implement an error log ring buffer that stores the last 8 error codes with timestamps
-

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day22
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day22/main.cpp](#)

